LAPORAN TUGAS KELOMPOK

MATA KULIAH KEAMANAN INFORMASI

FisioVault

Aplikasi Web Penyimpanan Dokumen Fisioterapi Pasien

dengan Perbandingan Algoritma Enkripsi AES, DES, dan RC4

Disusun oleh:

Nama	**NRP**
Rayyan Muhtar Ali	5054251051
Muhammad Farid Wijdan	5054251042

INSTITUT TEKNOLOGI SEPULUH NOPEMBER

SURABAYA

2026

DAFTAR ISI

BAB I PENDAHULUAN 4
1.1 Latar Belakang 4
1.2 Rumusan Masalah 4
1.3 Tujuan 4
1.4 Batasan Masalah 4
BAB II DASAR TEORI 6
2.1 Advanced Encryption Standard (AES) dan Mode GCM 6
2.2 Data Encryption Standard (DES) dan Mode CBC 6
2.3 RC4 6
2.4 Hashing Password dengan Argon2id 7
2.5 Envelope Encryption 7
BAB III PERANCANGAN DAN IMPLEMENTASI 8
3.1 Arsitektur Sistem 8
3.2 Desain Kriptografi 9
3.3 Keamanan Aplikasi 11
3.4 Antarmuka Pengguna 11
BAB IV PENGUJIAN DAN ANALISIS 12
4.1 Pengujian Fungsional 12
4.2 Metodologi Benchmark 12
4.3 Hasil Benchmark 13
4.4 Analisis Kinerja 14
4.5 Perbandingan Keamanan 15
BAB V KESIMPULAN DAN SARAN 16
5.1 Kesimpulan 16
5.2 Saran 16
DAFTAR PUSTAKA 17

LAMPIRAN 18
A. Repositori Kode 18
B. Cara Menjalankan 18
C. Log Percakapan dengan Asisten AI 18

BAB I PENDAHULUAN

1.1 Latar Belakang

Pasien fisioterapi sering berpindah fasilitas kesehatan (faskes), baik karena rujukan, pindah domisili, maupun perubahan jaminan kesehatan. Setiap kali berpindah, pasien perlu membawa dokumen terapinya: data pribadi, scan KTP dan kartu BPJS, surat rujukan, hasil rontgen, program latihan, hingga video rekaman progres gerak. Dokumen-dokumen ini termasuk data kesehatan yang sensitif, sehingga penyimpanannya harus terlindungi dari akses pihak yang tidak berhak.

FisioVault adalah aplikasi web yang memungkinkan pasien menyimpan sendiri dokumen terapinya secara terenkripsi. Untuk keperluan studi kriptografi, setiap dokumen dienkripsi dengan tiga algoritma sekaligus, yaitu **AES**, **DES**, dan **RC4**, sehingga ketiganya dapat dibandingkan pada data yang sama dari sisi kinerja maupun keamanan.

1.2 Rumusan Masalah

1. Bagaimana merancang aplikasi penyimpanan dokumen pasien yang tidak menyimpan data sensitif dalam bentuk plaintext?
2. Bagaimana menerapkan AES, DES, dan RC4 secara benar (mode operasi, IV, dan kunci) pada aplikasi tersebut, termasuk untuk file video berukuran besar?
3. Bagaimana perbandingan kinerja (waktu, throughput, memori, overhead ukuran) dan keamanan ketiga algoritma tersebut?

1.3 Tujuan

1. Membangun aplikasi web FisioVault dengan registrasi/login, unggah, dan unduh dokumen terenkripsi.
2. Mengimplementasikan AES-256-GCM, DES-CBC, dan RC4 dengan kunci acak per file dan enkripsi berbasis stream.
3. Mengukur dan menganalisis kinerja serta keamanan ketiga algoritma untuk memberi rekomendasi algoritma yang layak dipakai.

1.4 Batasan Masalah

- Jenis data: data pribadi (form), gambar JPG/PNG, dokumen PDF/DOCX, dan video MP4 hingga 1 GB.

- Ketiga algoritma dijalankan melalui modul `crypto` Node.js dengan backend OpenSSL yang sama agar perbandingan adil.
- Mode ECB tidak digunakan.
- Aplikasi dijalankan dan diuji secara lokal pada satu mesin (laptop).

BAB II DASAR TEORI

2.1 Advanced Encryption Standard (AES) dan Mode GCM

AES adalah block cipher berukuran blok 128 bit dengan panjang kunci 128, 192, atau 256 bit yang distandarkan NIST dalam FIPS 197. AES-256 menggunakan 14 ronde transformasi SubBytes, ShiftRows, MixColumns, dan AddRoundKey. Prosesor modern menyediakan instruksi khusus AES-NI sehingga AES dapat dieksekusi sangat cepat.

Galois/Counter Mode (GCM, NIST SP 800-38D) adalah mode operasi AEAD (Authenticated Encryption with Associated Data). GCM mengenkripsi dengan mode counter (CTR) sehingga tidak memerlukan padding, sekaligus menghasilkan authentication tag (umumnya 128 bit) melalui fungsi GHASH. Tag ini memungkinkan penerima mendeteksi setiap perubahan pada ciphertext. Syarat utamanya, pasangan kunci dan IV (nonce) tidak boleh dipakai ulang.

2.2 Data Encryption Standard (DES) dan Mode CBC

DES adalah block cipher berstruktur Feistel 16 ronde dengan ukuran blok 64 bit dan kunci 64 bit, dengan 56 bit efektif (8 bit paritas). DES ditarik sebagai standar oleh NIST pada 2005 (FIPS 46-3) karena kunci 56 bit dapat di-brute-force; mesin EFF Deep Crack memecahkannya dalam kurang dari tiga hari pada 1998. Ukuran blok 64 bit juga membuatnya rentan terhadap serangan birthday Sweet32 bila satu kunci dipakai untuk data yang sangat besar.

Cipher Block Chaining (CBC) meng-XOR setiap blok plaintext dengan blok ciphertext sebelumnya sebelum dienkripsi; blok pertama di-XOR dengan IV. CBC memerlukan padding (PKCS#7) agar panjang data kelipatan ukuran blok, dan IV harus acak serta tidak dapat ditebak. CBC hanya memberi kerahasiaan, tanpa integritas.

Mode ECB (Electronic Codebook) tidak digunakan karena mengenkripsi blok plaintext yang sama menjadi blok ciphertext yang sama, sehingga pola data (misalnya pada gambar KTP) tetap terlihat.

2.3 RC4

RC4 adalah stream cipher yang membangkitkan keystream pseudo-acak dari kunci (40–2048 bit) melalui Key Scheduling Algorithm (KSA) dan Pseudo-Random Generation Algorithm (PRGA). Ciphertext diperoleh dengan meng-XOR plaintext dengan keystream. RC4 tidak memiliki IV, sehingga pemakaian kunci yang sama untuk dua pesan menghasilkan $C_1 \oplus C_2 =$

$P_1 \oplus P_2$. Keystream RC4 juga diketahui bias, terutama pada byte-byte awal, yang dimanfaatkan dalam serangan terhadap WEP dan TLS. RFC 7465 (2015) melarang RC4 dipakai dalam TLS.

2.4 Hashing Password dengan Argon2id

Argon2 adalah pemenang Password Hashing Competition (2015) dan distandarkan dalam RFC 9106. Varian Argon2id bersifat memory-hard, yaitu setiap perhitungan hash membutuhkan memori dalam jumlah besar, sehingga brute-force dengan GPU atau ASIC menjadi mahal. OWASP merekomendasikan Argon2id untuk penyimpanan password.

2.5 Envelope Encryption

Envelope encryption adalah teknik manajemen kunci dengan dua lapis kunci: data dienkripsi dengan kunci data (data key), lalu kunci data dienkripsi ("dibungkus") dengan kunci induk (master key). Hanya kunci data terbungkus yang disimpan bersama data. Teknik ini dipakai oleh layanan Key Management Service seperti AWS KMS dan Google Cloud KMS.

BAB III PERANCANGAN DAN IMPLEMENTASI

3.1 Arsitektur Sistem

Komponen	Teknologi	Peran
Server web	Node.js 24 + Express 5	API registrasi/login, unggah, unduh
Database	SQLite (better-sqlite3)	User, sesi, metadata file (terenkripsi)
Kriptografi	Modul crypto Node.js (OpenSSL 3.5.6), argon2	AES, DES, RC4, Argon2id
Penyimpanan file	Folder storage/ di luar folder publik	Hanya ciphertext; nama file berupa UUID
Antarmuka	HTML, CSS, JavaScript (tanpa framework)	Login, unggah, data pribadi, daftar & unduh dokumen

Tabel 3.1 Komponen sistem FisioVault

Alur unggah dan unduh dokumen ditunjukkan pada diagram berikut.

```
UNGGAH
Browser --multipart--> busboy (stream, tanpa file sementara)
                         |-> SHA-256 (untuk verifikasi saat unduh)
                         |-> AES-256-GCM -> storage/<uuid>.aes
                         |-> DES-CBC     -> storage/<uuid>.des
                         \-> RC4         -> storage/<uuid>.rc4
Kunci per file dibungkus master key -> disimpan di DB bersama IV, tag, metadata terenkripsi

UNDUH
storage/<uuid>.<algo> -> dekripsi (stream) -> verifikasi SHA-256 -> Browser
                                               (chunk terakhir ditahan sampai hash cocok)
```

Gambar 3.1 Alur unggah dan unduh dokumen

Method	Path	Fungsi
POST	/auth/register	Daftar akun (username + password)
POST	/auth/login	Login, membuat cookie sesi
POST	/auth/logout	Menghapus sesi
GET	/auth/me	Info user yang sedang login
POST	/files	Unggah file JPG/PNG/PDF/DOCX/MP4 (maks. 1 GB)
POST	/files/form	Simpan data pribadi (nama, tgl lahir, HP, alamat, BPJS, diagnosis)
GET	/files	Daftar file milik sendiri
GET	/files/:id/download?algo=aes|des|rc4	Unduh dan dekripsi dengan algoritma pilihan
DELETE	/files/:id	Hapus file (DB dan ketiga ciphertext)

Tabel 3.2 Endpoint API

3.2 Desain Kriptografi

3.2.1 Parameter Algoritma

Aspek	AES-256-GCM	DES-CBC	RC4
Jenis	Block cipher, mode AEAD	Block cipher, mode CBC	Stream cipher
Ukuran kunci	256 bit	64 bit (efektif 56 bit)	128 bit
Ukuran blok	128 bit	64 bit	–
IV / nonce	12 byte acak per file	8 byte acak per file	Tidak ada (kunci acak per file)
Padding	Tidak perlu	PKCS#7	Tidak perlu
Integritas bawaan	Ya (auth tag 16 byte)	Tidak	Tidak
Status	Standar (NIST SP 800-38D)	Ditarik NIST (2005)	Dilarang di TLS (RFC 7465)

Tabel 3.3 Parameter ketiga algoritma

Alasan pemilihan parameter:

- **AES-256-GCM** dipilih karena memberi kerahasiaan sekaligus integritas. IV 12 byte adalah panjang yang direkomendasikan NIST. IV dibangkitkan di dalam fungsi enkripsi sehingga pemanggil tidak mungkin memakai ulang IV, karena pemakaian ulang IV pada GCM membocorkan plaintext dan memungkinkan pemalsuan tag.
- **DES-CBC** dipilih karena ECB dilarang. IV acak per file mencegah dua file berawalan sama menghasilkan ciphertext berawalan sama. Padding PKCS#7 ditangani OpenSSL, bukan kode buatan sendiri.
- **RC4** tidak memiliki IV, sehingga satu-satunya cara memperoleh keystream unik adalah kunci acak baru per file. RC4 dipakai dalam bentuk standar (tanpa RC4-drop) agar yang diuji adalah algoritma aslinya.

3.2.2 Satu Implementasi Kriptografi untuk Ketiga Algoritma

Pada OpenSSL 3, DES dan RC4 dipindahkan ke legacy provider yang tidak aktif secara default, sehingga muncul galat "unsupported". Kelompok memilih mengaktifkan legacy provider dengan flag `--openssl-legacy-provider` dibanding memakai pustaka JavaScript murni. Alasannya, dengan pustaka JavaScript benchmark menjadi tidak adil: AES berjalan pada kode C dengan akselerasi perangkat keras, sedangkan DES dan RC4 berjalan di JavaScript. Saat server dinyalakan, aplikasi memeriksa ketersediaan DES dan RC4 dan langsung berhenti dengan pesan yang jelas bila flag lupa dipasang.

3.2.3 Manajemen Kunci

- Setiap file mendapat kunci data acak sendiri untuk masing-masing algoritma.
- Kunci data dibungkus dengan master key AES-256-GCM yang disimpan di file `.env` (tidak masuk repositori). Hanya kunci terbungkus yang disimpan di database.
- Saat membungkus, ID file dan nama algoritma dipakai sebagai AAD (additional authenticated data), sehingga kunci milik satu file tidak dapat ditempelkan ke baris file lain.

Dibanding kunci yang diturunkan dari password pengguna, envelope encryption tidak membuat file hilang ketika pengguna lupa atau mengganti password, dan server tidak perlu menyimpan password atau kunci turunan selama sesi. Kelemahannya, pihak yang menguasai server sekaligus file `.env` dapat membuka semua file; di lingkungan produksi master key sebaiknya disimpan di HSM atau KMS.

3.2.4 Tidak Ada Plaintext Sensitif di Penyimpanan

Lokasi	Yang disimpan
Folder storage/	Hanya ciphertext; nama file berupa UUID acak
Tabel files	Kunci terbungkus, IV, auth tag, ukuran, dan metadata terenkripsi (nama file asli, MIME, SHA-256 plaintext)
Tabel users	Username dan hash Argon2id (bukan password)
Tabel sessions	SHA-256 dari token sesi (bukan token aslinya)
File sementara	Tidak ada; unggahan di-stream langsung ke cipher
Log	Hanya pesan galat, tanpa isi data

Tabel 3.4 Isi setiap lokasi penyimpanan

Hash SHA-256 plaintext sengaja ikut dienkripsi karena, untuk data berentropi rendah seperti form data pribadi, hash yang terbuka dapat ditebak dengan mencoba kombinasi nama dan tanggal lahir.

3.2.5 Verifikasi Integritas saat Unduh

Saat unduh, server menghitung SHA-256 hasil dekripsi dan menahan potongan data terakhir sampai hash (dan tag GCM untuk AES) terbukti cocok. Jika tidak cocok, koneksi diputus sehingga pengguna tidak pernah menerima file rusak secara utuh. Dengan cara ini DES dan RC4 juga memperoleh pemeriksaan integritas, tetapi di tingkat aplikasi, bukan dari algoritmanya. Perbedaan ini terlihat pada log pengujian:

```
aes : Unsupported state or unable to authenticate data    <- ditolak oleh GCM
```

```
des : Integritas gagal: SHA-256 hasil dekripsi tidak cocok <- ditolak oleh aplikasi
rc4 : Integritas gagal: SHA-256 hasil dekripsi tidak cocok <- ditolak oleh aplikasi
```

3.2.6 Enkripsi Berbasis Stream

Seluruh fungsi enkripsi menerima masukan dan keluaran berupa stream dengan potongan 64 KB. Satu stream unggahan dicabangkan ke ketiga cipher secara paralel, sehingga file hanya dibaca sekali, tidak pernah ditulis ke disk dalam bentuk plaintext, dan tidak pernah dimuat utuh ke memori. Hal ini penting untuk video berukuran ratusan MB.

3.3 Keamanan Aplikasi

Ancaman	**Mitigasi**
Password bocor dari database	Argon2id (memory-hard, 64 MB per hash)
Penyalahgunaan hashing (DoS)	Panjang password dibatasi 8–128 karakter
Token sesi bocor dari database	Hanya SHA-256 token yang disimpan
Pencurian cookie melalui XSS	Cookie HttpOnly, Content-Security-Policy, tampilan memakai textContent
CSRF	Cookie SameSite=Lax
Penyadapan	Cookie Secure pada mode produksi (HTTPS)
Session fixation	Token baru setiap login
User enumeration	Pesan dan waktu respons login sama untuk username atau password salah
IDOR (akses file orang lain)	Middleware pemeriksa kepemilikan; file orang lain dijawab 404, bukan 403
Unggahan berbahaya	Daftar ekstensi yang diizinkan, MIME ditentukan server, batas 1 GB
Clickjacking	Header frame-ancestors none
File sensitif tersimpan di cache	Header Cache-Control: no-store

Tabel 3.5 Ancaman dan mitigasi

3.4 Antarmuka Pengguna

Antarmuka dibuat sederhana dengan gaya aplikasi rekam kesehatan: latar putih, aksen hijau toska, kartu, ikon besar, dan tata letak yang ramah perangkat seluler. Halaman terdiri atas tampilan login/daftar dan dasbor dengan dua aksi utama (unggah dokumen dan isi data pribadi), daftar dokumen, serta pilihan algoritma saat mengunduh (AES-256-GCM sebagai pilihan yang disarankan; DES dan RC4 sebagai pembanding). Unggahan menampilkan progress bar agar nyaman untuk video berukuran besar.

BAB IV PENGUJIAN DAN ANALISIS

4.1 Pengujian Fungsional

Pengujian otomatis dijalankan dengan Node.js test runner (`npm test`). Sebanyak **35 tes dijalankan dan semuanya lulus.**

Kelompok	Jumlah	Yang dibuktikan
AES-256-GCM	5	Round-trip 5 MB identik (SHA-256), file kosong, IV unik, ciphertext diubah 1 bit ditolak, kunci salah ditolak
DES-CBC	5	Round-trip identik, ukuran padding PKCS#7 benar, IV unik, kunci salah tidak membocorkan plaintext, ciphertext diubah tidak terdeteksi
RC4	6	Round-trip identik, file kosong, kunci unik per file, kunci salah tidak galat, malleability, bahaya pemakaian ulang kunci
Autentikasi	9	Hash Argon2id, username unik, validasi input, pesan login seragam, atribut cookie, token tidak tersimpan mentah, logout, sesi kedaluwarsa, kepemilikan file
Unggah/unduh	10	Enkripsi 3x, tanpa plaintext di storage/DB, unduh 3 algoritma identik, isolasi antar-user, tipe file ditolak, perubahan terdeteksi, form data pribadi, kunci tertukar ditolak, hapus file

Tabel 4.1 Ringkasan pengujian fungsional

Tiga hasil pengujian yang paling relevan untuk perbandingan algoritma:

1. **Integritas.** Perubahan 1 bit pada ciphertext langsung ditolak oleh AES-GCM, sedangkan DES-CBC dan RC4 menerimanya tanpa galat dengan isi yang rusak.
2. **Malleability RC4.** Teks "Sesi terapi: 2x/minggu" dapat diubah menjadi "9x/minggu" tanpa mengetahui kunci, cukup dengan meng-XOR satu byte ciphertext.
3. **Pemakaian ulang kunci RC4.** Jika dua file dienkripsi dengan kunci yang sama, berlaku $C_1 \oplus C_2 = P_1 \oplus P_2$; mengetahui isi satu file langsung membuka isi file lainnya. Ini menjadi dasar aturan kunci acak per file.

4.2 Metodologi Benchmark

- Benchmark dijalankan dengan `npm run bench` dan memakai modul enkripsi yang sama dengan aplikasi.
- Data uji berupa byte acak (bukan data pasien) dengan ukuran yang mewakili jenis dokumen: 1 KB (form), 100 KB (scan KTP/BPJS), 1 MB (PDF), 10 MB (rontgen/DOCX), dan 100 MB (video).

- Masukan di-stream dalam potongan 64 KB dan keluaran dibuang agar kecepatan disk tidak ikut terukur.
- Setiap ukuran diulang 5–200 kali (file kecil lebih sering) dengan satu putaran pemanasan yang tidak dihitung; nilai yang dilaporkan adalah median.
- Memori puncak (RSS) diukur pada file 256 MB di proses terpisah untuk setiap algoritma, dibandingkan dengan cara naif yang membaca file utuh ke memori.
- Integritas diperiksa dengan membandingkan SHA-256 hasil dekripsi dengan plaintext asli.

Lingkungan pengujian: 12th Gen Intel(R) Core(TM) i7-12700H (20 thread), RAM 15,7 GB, Windows 11 (10.0.26200), Node.js v24.16.0, OpenSSL 3.5.6.

4.3 Hasil Benchmark

4.3.1 Waktu Enkripsi dan Dekripsi

Ukuran	AES-256-GCM	DES-CBC	RC4
1 KB (form data pribadi)	0,17 / 0,16	0,14 / 0,13	0,09 / 0,09
100 KB (scan KTP/BPJS)	0,20 / 0,19	1,90 / 1,68	0,52 / 0,52
1 MB (PDF rujukan)	1,11 / 1,13	17,15 / 15,19	4,13 / 4,07
10 MB (rontgen/DOCX besar)	13,31 / 10,68	513,02 / 405,44	51,05 / 53,06
100 MB (video latihan)	366,63 / 338,81	4925,12 / 5077,52	513,01 / 910,98

Tabel 4.2 Waktu enkripsi / dekripsi (median, milidetik)

4.3.2 Throughput

Ukuran	AES-256-GCM	DES-CBC	RC4
1 KB (form data pribadi)	5,7 / 6,2	7,1 / 7,6	11,3 / 11,3
100 KB (scan KTP/BPJS)	479,2 / 512,9	51,3 / 58,1	189,2 / 189,1
1 MB (PDF rujukan)	900,8 / 887,3	58,3 / 65,8	242,4 / 245,6
10 MB (rontgen/DOCX besar)	751,2 / 936,2	19,5 / 24,7	195,9 / 188,5
100 MB (video latihan)	272,8 / 295,2	20,3 / 19,7	194,9 / 109,8

Tabel 4.3 Throughput enkripsi / dekripsi (MB/s)

4.3.3 Memori Puncak

Algoritma	Cara	Enkripsi (MB)	Dekripsi (MB)
AES-256-GCM	stream (dipakai aplikasi)	33,6	18,5
DES-CBC	stream (dipakai aplikasi)	33,7	34,9

Algoritma	Cara	Enkripsi (MB)	Dekripsi (MB)
RC4	stream (dipakai aplikasi)	34,4	36,9
AES-256-GCM	buffer utuh (pembanding)	768,1	512,0

Tabel 4.4 Memori puncak untuk file 256 MB

4.3.4 Overhead Ukuran

Algoritma	Overhead per file	Komponen
AES-256-GCM	28 byte	IV 12 byte + auth tag 16 byte
DES-CBC	9–16 byte	IV 8 byte + padding 1–8 byte
RC4	0 byte	–

Tabel 4.5 Overhead ukuran

Untuk file berukuran 100 KB atau lebih, overhead ketiga algoritma kurang dari 0,03%.

4.3.5 Integritas

Round-trip ketiga algoritma pada kelima ukuran menghasilkan SHA-256 yang identik dengan plaintext asli (15/15 berhasil).

4.4 Analisis Kinerja

1. **AES-256-GCM paling cepat** untuk file 100 KB ke atas: pada ukuran 1 MB sekitar 15 kali lebih cepat dari DES dan 3–4 kali lebih cepat dari RC4. Penyebabnya adalah instruksi perangkat keras AES-NI (dan PCLMULQDQ untuk GHASH pada GCM) di prosesor modern, sedangkan DES dan RC4 dihitung dengan instruksi umum. Anggapan bahwa algoritma lama lebih "ringan" tidak berlaku pada perangkat keras saat ini.
2. **DES paling lambat** karena 16 ronde Feistel untuk setiap blok yang hanya 64 bit, ditambah mode CBC yang proses enkripsinya tidak dapat diparalelkan.
3. **Pada ukuran 1 KB urutannya terbalik** (RC4 lebih cepat dari DES, DES lebih cepat dari AES), tetapi semuanya di bawah 0,2 ms. Pada ukuran sekecil ini waktu didominasi biaya pembuatan stream dan inisialisasi cipher, bukan algoritmanya, sehingga baris ini tidak dapat dipakai untuk menyimpulkan kecepatan algoritma.
4. **Stream membuat pemakaian memori konstan** (sekitar 34 MB) berapa pun ukuran filenya, sedangkan membaca file utuh membutuhkan sekitar tiga kali ukuran file (768 MB untuk file 256 MB). Untuk video berukuran ratusan MB dengan banyak pengguna, cara buffer tidak layak.

5. **Overhead ukuran tidak menjadi pembeda**; pembeda utamanya adalah keamanan dan kecepatan.

Catatan validitas: simpangan baku pada ukuran 10–100 MB cukup besar (misalnya DES 10 MB: 513 ± 163 ms). Kemungkinan penyebabnya adalah penurunan kecepatan CPU laptop karena mode daya atau suhu, proses lain yang berjalan, serta arsitektur prosesor dengan inti performa dan inti efisiensi yang kecepatannya berbeda. Pola perbandingan antaralgoritma tetap konsisten pada setiap pengulangan.

4.5 Perbandingan Keamanan

Aspek	AES-256-GCM	DES-CBC	RC4
Brute-force kunci	2^256, tidak mungkin	2^56, sudah dipecahkan (EFF Deep Crack, 1998)	2^128, kuat dari sisi ukuran kunci
Kelemahan struktural	Tidak ada yang praktis bila IV tidak dipakai ulang	Blok 64 bit, rentan Sweet32 setelah ±32 GB data dengan satu kunci	Keystream bias, dimanfaatkan pada serangan WEP dan TLS
Integritas bawaan	Ya (auth tag)	Tidak; rentan padding oracle bila galat padding terlihat	Tidak; malleable bit per bit
Risiko salah pakai	IV dipakai ulang berakibat fatal	IV dapat ditebak membocorkan pola	Kunci dipakai ulang: $C_1 \oplus C_2 = P_1 \oplus P_2$
Rekomendasi	Layak dipakai	Tidak untuk data baru	Tidak untuk data baru

Tabel 4.6 Perbandingan keamanan

BAB V KESIMPULAN DAN SARAN

5.1 Kesimpulan

1. Aplikasi FisioVault berhasil dibangun: pasien dapat mendaftar, login, mengunggah dokumen (data pribadi, gambar, PDF, DOCX, dan video), lalu mengunduhnya kembali dalam keadaan utuh. Tidak ada plaintext data sensitif yang tersimpan di database, folder penyimpanan, log, maupun file sementara.
2. **AES-256-GCM unggul di semua aspek yang diukur**: paling aman (kunci 256 bit dan integritas bawaan) sekaligus paling cepat untuk dokumen berukuran 100 KB ke atas berkat akselerasi perangkat keras.
3. **DES tidak layak** untuk data kesehatan karena kunci 56 bit dapat di-brute-force, blok 64 bit rentan Sweet32, dan kecepatannya paling rendah (sekitar 20–60 MB/s).
4. **RC4 lebih cepat dari DES** tetapi tidak memiliki integritas, bersifat malleable, dan keystream-nya bias; kesalahan memakai ulang kunci langsung membocorkan data.
5. Enkripsi berbasis stream membuat pemakaian memori tetap rendah (sekitar 34 MB) bahkan untuk file ratusan MB, sesuai kebutuhan penyimpanan video latihan.

5.2 Saran

- Untuk penggunaan nyata, FisioVault cukup memakai AES-256-GCM saja; DES dan RC4 di aplikasi ini hanya untuk keperluan perbandingan.
- Menyimpan master key di HSM/KMS dan menerapkan rotasi kunci.
- Menambahkan rate limiting pada login untuk mencegah brute-force online.
- Memeriksa isi file berdasarkan magic bytes, bukan hanya ekstensi.
- Memakai format chunked AEAD (tag per potongan) agar kerusakan pada video berukuran sangat besar terdeteksi lebih awal.
- Menambah fitur berbagi dokumen ke faskes melalui tautan sementara.
- Menjalankan benchmark pada beberapa mesin (termasuk server dan prosesor ARM) untuk hasil yang lebih umum.

DAFTAR PUSTAKA

1. National Institute of Standards and Technology. (2001, diperbarui 2023). FIPS 197: Advanced Encryption Standard (AES).
2. Dworkin, M. (2007). NIST SP 800-38D: Recommendation for Block Cipher Modes of Operation: Galois/Counter Mode (GCM) and GMAC. NIST.
3. Dworkin, M. (2001). NIST SP 800-38A: Recommendation for Block Cipher Modes of Operation: Methods and Techniques. NIST.
4. National Institute of Standards and Technology. (1999, ditarik 2005). FIPS 46-3: Data Encryption Standard (DES).
5. Popov, A. (2015). RFC 7465: Prohibiting RC4 Cipher Suites. IETF.
6. Bhargavan, K., & Leurent, G. (2016). On the Practical (In-)Security of 64-bit Block Ciphers: Collision Attacks on HTTP over TLS and OpenVPN. ACM CCS 2016.
7. AlFardan, N., Bernstein, D. J., Paterson, K. G., Poettering, B., & Schuldt, J. C. N. (2013). On the Security of RC4 in TLS. USENIX Security Symposium 2013.
8. Electronic Frontier Foundation. (1998). Cracking DES: Secrets of Encryption Research, Wiretap Politics & Chip Design. O'Reilly.
9. Biryukov, A., Dinu, D., Khovratovich, D., & Josefsson, S. (2021). RFC 9106: Argon2 Memory-Hard Function for Password Hashing and Proof-of-Work Applications. IETF.
10. OWASP Foundation. Password Storage Cheat Sheet dan Session Management Cheat Sheet. https://cheatsheetseries.owasp.org
11. OpenSSL Project. OpenSSL 3.0 Migration Guide: Legacy Provider. https://docs.openssl.org

LAMPIRAN

A. Repositori Kode

Kode sumber lengkap FisioVault tersedia di: **https://github.com/rayyan129/fisiovault**

Folder / file	Isi
src/crypto/	Implementasi AES-256-GCM, DES-CBC, RC4, dan pembungkus kunci
src/routes/, src/middleware/, src/auth/	Registrasi/login, sesi, unggah/unduh, pemeriksa kepemilikan
src/services/vault.js	Enkripsi tiga algoritma sekaligus dan verifikasi integritas
public/	Antarmuka web
test/	35 tes otomatis
bench/	Skrip benchmark dan hasilnya (bench/results/)

B. Cara Menjalankan

1. Pasang Node.js versi 22.9 atau lebih baru (disarankan v24 LTS).
2. Jalankan `npm install` di folder proyek.
3. Salin `.env.example` menjadi `.env`, lalu isi `FISIOVAULT_MASTER_KEY` dengan 64 karakter hex hasil perintah `node -e "console.log(require('crypto').randomBytes(32).toString('hex'))"`.
4. Jalankan `npm test` untuk pengujian, `npm run bench` untuk benchmark, atau `npm start` untuk menjalankan aplikasi.
5. Buka http://localhost:3000 di browser.

C. Log Percakapan dengan Asisten AI

Proyek ini dikerjakan dengan bantuan asisten AI **Claude Code** (model Claude Opus 5.5, Anthropic). Transkrip lengkap percakapan selama pengerjaan, berisi pesan kelompok, jawaban asisten, dan daftar perintah yang dijalankan, disertakan di repositori agar dapat diperiksa:

https://github.com/rayyan129/fisiovault/blob/dev/docs/percakapan-claude-code.md

Pengerjaan dilakukan bertahap sesuai urutan langkah di file CLAUDE.md (setup, kriptografi, autentikasi, unggah/unduh, benchmark, bahan laporan, antarmuka). Keputusan desain, seperti pengaktifan OpenSSL legacy provider, diambil oleh kelompok setelah asisten menjelaskan pilihan-pilihannya.